CourseHub API

Evidencias de funcionamiento

Aplicaciones para el servidor web
Semanas 5 y 6 · Sesiones 9, 10, 11 y 12

Cada caso muestra la petición enviada, la respuesta recibida y el código HTTP devuelto por la API.

COURSEHUB API - EVIDENCIAS DEL PROYECTO INTEGRADOR SEMANA 5 · Estudiantes y matriculas persistentes en PostgreSQL

Entorno de la demostracion

```
--------------------------
API            NestJS sobre http://localhost:3000
Base de datos  PostgreSQL 18, base "coursehub"
Tablas         courses, students, enrollments
Herramienta    curl (se transcriben peticion y respuesta completas)
```

Todas las tablas se vaciaron antes de empezar, por eso los

identificadores comienzan en 1.

Codigos HTTP elegidos y su justificacion

```
--------------------------------------
201  Creacion correcta de curso, estudiante o matricula.
200  Consulta o cancelacion correcta.
400  El estudiante existe pero esta inactivo. Es un dato invalido
     enviado por el cliente, no un conflicto con el estado del
     recurso: el cliente debe corregir la peticion.
404  El estudiante o el curso referenciado no existe, o la
     matricula que se intenta cancelar ya no esta.
409  Conflicto con el estado actual de los datos: la pareja
     estudiante-curso ya existe, o el correo ya esta registrado.
```

Resumen de los casos demostrados

```
--------------------------------
CASO 1  Crear un curso y un estudiante activo ..... 201
CASO 2  Crear una matricula valida ................ 201
CASO 3  Reiniciar la API y consultar la misma matricula .... 200
CASO 4  Matricula duplicada ....................... 409
CASO 5  Matricular a un estudiante inactivo ....... 400
CASO 6  Filtrar matriculas por estudiante o por curso ...... 200
CASO 7  Cancelar la matricula y comprobar que no esta ...... 200 y 404
ANEXO A Correo de estudiante duplicado ..................... 409
ANEXO B Estudiante o curso inexistente ..................... 404
```

ANEXO C Estructura relacional creada en PostgreSQL

CASO 1 - Crear un curso y un estudiante activo

PETICIÓN

```
POST /courses
Content-Type: application/json

{"title":"NestJS con PostgreSQL","level":"intermediate"}
```

RESPUESTA

```
{
  "id": 1,
  "title": "NestJS con PostgreSQL",
  "level": "intermediate"
}
```

HTTP 201

PETICIÓN

```
POST /courses
```

```
Content-Type: application/json


{"title":"Fundamentos de TypeScript","level":"beginner"}
```

RESPUESTA

```
{
  "id": 2,
  "title": "Fundamentos de TypeScript",
  "level": "beginner"
}
```

HTTP 201

PETICIÓN

```
POST /students
Content-Type: application/json


{"name":"Ana Perez","email":"ana@example.com","age":20,"career":"Software
Engineering","semester":4,"isActive":true}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Perez",
  "email": "ana@example.com",
  "age": 20,
  "career": "Software Engineering",
  "semester": 4,
  "isActive": true
}
```

HTTP 201

CASO 2 - Crear una matricula valida

PETICIÓN

```
POST /enrollments
Content-Type: application/json


{"studentId":1,"courseId":1}
```

RESPUESTA

```
{
  "id": 1,
  "studentId": 1,
  "courseId": 1,
  "student": {
    "id": 1,
    "name": "Ana Perez",
    "career": "Software Engineering",
    "semester": 4,
    "isActive": true
  },
  "course": {
    "id": 1,
    "title": "NestJS con PostgreSQL",
    "level": "intermediate"
  }
}
```

HTTP 201

CASO 3 - Reiniciar la API y consultar la misma matricula

(La API fue detenida con Ctrl+C y vuelta a iniciar con npm run start)

PETICIÓN

```
GET /enrollments/
```

RESPUESTA


```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Perez",
      "career": "Software Engineering",
      "semester": 4,
      "isActive": true
    },
    "course": {
      "id": 1,
      "title": "NestJS con PostgreSQL",
      "level": "intermediate"
    }
  }
]
```


HTTP 200

PETICIÓN

```
GET /students/1/enrollments
```

RESPUESTA


```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Perez",
      "career": "Software Engineering",
      "semester": 4,
      "isActive": true
    },
    "course": {
      "id": 1,
      "title": "NestJS con PostgreSQL",
      "level": "intermediate"
    }
  }
]
```


HTTP 200

El estudiante id 1 tambien sobrevivio al reinicio:

PETICIÓN

```
GET /students/1
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Perez",
  "email": "ana@example.com",
  "age": 20,
  "career": "Software Engineering",
  "semester": 4,
  "isActive": true
}
```

HTTP 200

CASO 4 - Matricula duplicada (409 Conflict)

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":1,"courseId":1}
```

RESPUESTA

```
{
  "message": "El estudiante ya está matriculado en este curso",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

CASO 5 - Matricular a un estudiante inactivo (400 Bad Request)

Primero se desactiva al estudiante:

PETICIÓN

```
PATCH /students/1/status
Content-Type: application/json

{"isActive":false}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Perez",
  "email": "ana@example.com",
  "age": 20,
  "career": "Software Engineering",
  "semester": 4,
  "isActive": false
}
```

HTTP 200

Y se intenta matricularlo en el curso 2:

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":1,"courseId":2}
```

RESPUESTA


```
{
  "message": "Un estudiante inactivo no puede matricularse",
  "error": "Bad Request",
  "statusCode": 400
}
```


HTTP 400

CASO 6 - Filtrar las matriculas por estudiante o por curso

PETICIÓN

```
GET /enrollments?studentId=1
```

RESPUESTA


```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Perez",
      "career": "Software Engineering",
      "semester": 4,
      "isActive": false
    },
    "course": {
      "id": 1,
      "title": "NestJS con PostgreSQL",
      "level": "intermediate"
    }
  }
]
```


HTTP 200

PETICIÓN

```
GET /enrollments?courseId=1
```

RESPUESTA


```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Perez",
      "career": "Software Engineering",
      "semester": 4,
      "isActive": false
    },
    "course": {
      "id": 1,
      "title": "NestJS con PostgreSQL",
      "level": "intermediate"
    }
  }
```

```
]
```

HTTP 200

PETICIÓN

```
GET /enrollments?studentId=1&courseId=1
```

RESPUESTA

```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Perez",
      "career": "Software Engineering",
      "semester": 4,
      "isActive": false
    },
    "course": {
      "id": 1,
      "title": "NestJS con PostgreSQL",
      "level": "intermediate"
    }
  }
]
```

HTTP 200

Filtro que no coincide con ninguna matricula:

PETICIÓN

```
GET /enrollments?courseId=2
```

RESPUESTA

```
[]
```

HTTP 200

Rutas anidadas equivalentes:

PETICIÓN

```
GET /courses/1/enrollments
```

RESPUESTA

```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Perez",
      "career": "Software Engineering",
      "semester": 4,
      "isActive": false
    },
    "course": {
      "id": 1,
      "title": "NestJS con PostgreSQL",
      "level": "intermediate"
    }
```

```
  }
]
```

HTTP 200

CASO 7 - Cancelar la matricula y comprobar que ya no se encuentra

PETICIÓN

```
DELETE /enrollments/1
```

RESPUESTA


```
{
  "id": 1,
  "studentId": 1,
  "courseId": 1,
  "student": {
    "id": 1,
    "name": "Ana Perez",
    "career": "Software Engineering",
    "semester": 4,
    "isActive": false
  },
  "course": {
    "id": 1,
    "title": "NestJS con PostgreSQL",
    "level": "intermediate"
  }
}
```


HTTP 200

La lista queda vacia:

PETICIÓN

```
GET /enrollments
```

RESPUESTA

```
[]
```

HTTP 200

Tampoco aparece en las rutas anidadas:

PETICIÓN

```
GET /students/1/enrollments
```

RESPUESTA

```
[]
```

HTTP 200

Y cancelarla de nuevo responde 404:

PETICIÓN

```
DELETE /enrollments/1
```

RESPUESTA


```
{
  "message": "Matrícula con id 1 no encontrada",
  "error": "Not Found",
  "statusCode": 404
}
```


HTTP 404

ANEXO A - Correo unico (validado en el servicio y en la base)

PETICIÓN

```
POST /students
Content-Type: application/json

{"name":"Otro Estudiante","email":"ana@example.com","age":22,"career":"Software
Engineering","semester":5,"isActive":true}
```

RESPUESTA

```
{
  "message": "Ya existe un estudiante con este correo",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

ANEXO B - Recursos inexistentes (404)

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":999,"courseId":1}
```

RESPUESTA

```
{
  "message": "Estudiante con id 999 no encontrado",
  "error": "Not Found",
  "statusCode": 404
}
```

HTTP 404

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":1,"courseId":999}
```

RESPUESTA

```
{
  "message": "Curso con id 999 no encontrado",
  "error": "Not Found",
  "statusCode": 404
}
```

HTTP 404

ANEXO C - Estructura relacional creada en PostgreSQL

Salida de \d enrollments en psql. Demuestra que las matriculas usan claves foraneas reales hacia students y courses, y no numeros sueltos, y que la pareja estudiante-curso tiene una restriccion unica en la propia base de datos.

```
                          Tabla «public.enrollments»
  Columna  |  Tipo   | Ordenamiento | Nulable  |               Por omisi
```

```
-----------+---------+--------------+----------+---------------------------------------
 id        | integer |              | not null | nextval('enrollments_id_seq'::regclass)
 studentId | integer |              | not null |
 courseId  | integer |              | not null |
```

Índices:

```
    "PK_7c0f752f9fb68bf6ed7367ab00f" PRIMARY KEY, btree (id)
    "UQ_enrollment_student_course" UNIQUE CONSTRAINT, btree ("studentId", "courseId")
```

Restricciones de llave foránea:

```
    "FK_60dd0ae4e21002e63a5fdefeec8" FOREIGN KEY ("courseId") REFERENCES courses(id) ON DELETE CASCADE
    "FK_bf3ba3dfa95e2df7388eb4589fd" FOREIGN KEY ("studentId") REFERENCES students(id) ON DELETE
CASCADE
```

La tabla students tiene ademas una restriccion UNIQUE sobre la

columna email, de modo que el correo unico se garantiza tanto en

el servicio como en PostgreSQL.

SESION 9 - ESTUDIANTES PERSISTENTES CON TYPEORM

La tabla students se vacio antes de empezar, por eso los

identificadores vuelven a comenzar en 1.

Resumen

```
-------
A  Crear estudiantes ..................................... 201
B  Filtros individuales y combinados ..................... 200
C  Actualizar un estudiante ............................... 200
D  Reiniciar la API y consultar .......................... 200
E  Correo repetido, detectado por el servicio ............. 409
F  No se elimina un estudiante inactivo ................... 409
G  Correo repetido, detectado por PostgreSQL .............. 409
```

SESION 9 - A - Crear estudiantes

PETICIÓN

```
POST /students
Content-Type: application/json


{"name":"Ana
Torres","email":"ana.torres@example.edu","age":20,"career":"Software","semester":4,"isActive":true}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software",
  "semester": 4,
  "isActive": true
}
```

HTTP 201

PETICIÓN

```
POST /students
Content-Type: application/json


{"name":"Luis
Mora","email":"luis.mora@example.edu","age":23,"career":"Redes","semester":7,"isActive":true}
```

RESPUESTA

```
{
  "id": 2,
  "name": "Luis Mora",
  "email": "luis.mora@example.edu",
  "age": 23,
  "career": "Redes",
  "semester": 7,
  "isActive": true
}
```

HTTP 201

SESION 9 - B - Filtros individuales y combinados

PETICIÓN

```
GET /students?career=Software
```

RESPUESTA

```
[
  {
    "id": 1,
    "name": "Ana Torres",
    "email": "ana.torres@example.edu",
    "age": 20,
    "career": "Software",
    "semester": 4,
    "isActive": true
  }
]
```

HTTP 200

PETICIÓN

```
GET /students?career=Software&isActive=true
```

RESPUESTA

```
[
  {
    "id": 1,
    "name": "Ana Torres",
    "email": "ana.torres@example.edu",
    "age": 20,
    "career": "Software",
    "semester": 4,
    "isActive": true
  }
]
```

HTTP 200

PETICIÓN

```
GET /students?semester=7
```

RESPUESTA

```
[
  {
    "id": 2,
    "name": "Luis Mora",
    "email": "luis.mora@example.edu",
    "age": 23,
    "career": "Redes",
    "semester": 7,
    "isActive": true
  }
]
```

HTTP 200

El texto false NO se convierte en booleano verdadero:

PETICIÓN

```
GET /students?isActive=false
```

RESPUESTA

```
[]
```

HTTP 200

SESION 9 - C - Actualizar

PETICIÓN

```
PATCH /students/1
Content-Type: application/json

{"semester":5,"career":"Software Engineering"}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software Engineering",
  "semester": 5,
  "isActive": true
}
```

HTTP 200

SESION 9 - D - Reiniciar la API y consultar

(La API fue detenida y vuelta a iniciar con npm run start)

PETICIÓN

```
GET /students/1
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software Engineering",
  "semester": 5,
  "isActive": true
}
```

HTTP 200

La actualizacion del paso C tambien sobrevivio: career y semester siguen cambiados.

PETICIÓN

```
GET /students
```

RESPUESTA

```
[
  {
    "id": 2,
    "name": "Luis Mora",
    "email": "luis.mora@example.edu",
    "age": 23,
    "career": "Redes",
    "semester": 7,
    "isActive": true
  },
  {
    "id": 1,
    "name": "Ana Torres",
```

```
    "email": "ana.torres@example.edu",
    "age": 20,
    "career": "Software Engineering",
    "semester": 5,
    "isActive": true
  }
]
```

HTTP 200

SESION 9 - E - Correo repetido (409 desde el servicio)

PETICIÓN

```
POST /students
Content-Type: application/json

{"name":"Otra
Persona","email":"ana.torres@example.edu","age":21,"career":"Redes","semester":3,"isActive":true}
```

RESPUESTA

```
{
  "message": "Ya existe un estudiante con este correo",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

Tambien al cambiar el correo de un estudiante por el de otro:

PETICIÓN

```
PATCH /students/2
Content-Type: application/json

{"email":"ana.torres@example.edu"}
```

RESPUESTA

```
{
  "message": "Ya existe un estudiante con este correo",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

SESION 9 - F - No se elimina un estudiante inactivo

PETICIÓN

```
PATCH /students/2/status
Content-Type: application/json

{"isActive":false}
```

RESPUESTA

```
{
  "id": 2,
  "name": "Luis Mora",
  "email": "luis.mora@example.edu",
  "age": 23,
  "career": "Redes",
  "semester": 7,
  "isActive": false
```

```
}
```

HTTP 200

PETICIÓN

```
DELETE /students/2
```

RESPUESTA

```
{
  "message": "No se puede eliminar un estudiante inactivo",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

Al reactivarlo si se puede eliminar:

PETICIÓN

```
PATCH /students/2/status
Content-Type: application/json

{"isActive":true}
```

RESPUESTA

```
{
  "id": 2,
  "name": "Luis Mora",
  "email": "luis.mora@example.edu",
  "age": 23,
  "career": "Redes",
  "semester": 7,
  "isActive": true
}
```

HTTP 200

PETICIÓN

```
DELETE /students/2
```

RESPUESTA

```
{
  "id": 2,
  "name": "Luis Mora",
  "email": "luis.mora@example.edu",
  "age": 23,
  "career": "Redes",
  "semester": 7,
  "isActive": true
}
```

HTTP 200

PETICIÓN

```
GET /students/2
```

RESPUESTA

```
{
  "message": "Estudiante con id 2 no encontrado",
  "error": "Not Found",
  "statusCode": 404
}
```

HTTP 404

SESION 9 - G - Correo repetido detectado por PostgreSQL

El servicio comprueba el correo antes de guardar, de modo que en condiciones normales el 409 lo produce esa comprobacion. Pero si dos solicitudes llegan a la vez, ambas pueden ver el correo libre y la que pierde la carrera choca contra la restriccion UNIQUE de la tabla.

Para demostrar que ese caso tampoco termina en un 500, se desactivo temporalmente la comprobacion previa del servicio y se repitio el POST con un correo ya registrado. El filtro global UniqueViolationFilter convirtio el error de integridad en un 409 con el mismo mensaje:

PETICIÓN

```
POST /students
Content-Type: application/json


{"name":"Impostor","email":"ana.torres@example.edu","age":22,"career":"Software","semester":5,"isActive
":true}
```

RESPUESTA

```
{
  "message": "Ya existe un estudiante con este correo",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

La comprobacion previa del servicio se restauro inmediatamente despues de esta prueba.

SESION 10 - MATRICULAS COMO RELACIONES PERSISTENTES

Las tres tablas se vaciaron antes de empezar.

Resumen

```
-------
A  Curso y estudiante persistentes ..... 201
B  Registrar la matricula ............... 201
C  Consultar con los datos relacionados ... 200
D  Pareja estudiante-curso repetida ..... 409
E  Estudiante inactivo en otro curso ..... 400
F  onDelete RESTRICT protege el vinculo ... 409
G  Reiniciar la API y cancelar la matricula ... 200
```

SESION 10 - A - Curso y estudiante persistentes

PETICIÓN

```
POST /courses
Content-Type: application/json

{"title":"Bases de datos relacionales","level":"intermediate"}
```

RESPUESTA

```
{
  "id": 1,
  "title": "Bases de datos relacionales",
  "level": "intermediate"
}
```

HTTP 201

PETICIÓN

```
POST /courses
Content-Type: application/json

{"title":"Introduccion a NestJS","level":"beginner"}
```

RESPUESTA

```
{
  "id": 2,
  "title": "Introduccion a NestJS",
  "level": "beginner"
}
```

HTTP 201

PETICIÓN

```
POST /students
Content-Type: application/json

{"name":"Ana
Torres","email":"ana.torres@example.edu","age":20,"career":"Software","semester":4,"isActive":true}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software",
  "semester": 4,
```

```
  "isActive": true
}
```

HTTP 201

SESION 10 - B - Registrar la matricula

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":1,"courseId":1}
```

RESPUESTA

```
{
  "id": 1,
  "studentId": 1,
  "courseId": 1,
  "student": {
    "id": 1,
    "name": "Ana Torres",
    "career": "Software",
    "semester": 4,
    "isActive": true
  },
  "course": {
    "id": 1,
    "title": "Bases de datos relacionales",
    "level": "intermediate"
  }
}
```

HTTP 201

SESION 10 - C - Consultar con los datos relacionados

PETICIÓN

```
GET /enrollments?studentId=1
```

RESPUESTA

```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Torres",
      "career": "Software",
      "semester": 4,
      "isActive": true
    },
    "course": {
      "id": 1,
      "title": "Bases de datos relacionales",
      "level": "intermediate"
    }
  }
]
```

HTTP 200

SESION 10 - D - Pareja repetida

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":1,"courseId":1}
```

RESPUESTA

```
{
  "message": "El estudiante ya está matriculado en este curso",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

SESION 10 - E - Estudiante inactivo en otro curso

PETICIÓN

```
PATCH /students/1/status
Content-Type: application/json

{"isActive":false}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software",
  "semester": 4,
  "isActive": false
}
```

HTTP 200

PETICIÓN

```
POST /enrollments
Content-Type: application/json

{"studentId":1,"courseId":2}
```

RESPUESTA

```
{
  "message": "Un estudiante inactivo no puede matricularse",
  "error": "Bad Request",
  "statusCode": 400
}
```

HTTP 400

SESION 10 - F - onDelete RESTRICT protege el vinculo

Un curso con matriculas no se puede eliminar.

NOTA: en la primera ejecucion este caso devolvio un 500 porque el

filtro solo contemplaba el codigo 23503 de PostgreSQL. Una clave

foranea con ON DELETE RESTRICT produce en realidad el codigo 23001.

Tras corregir el filtro, la respuesta es la siguiente:

PETICIÓN

```
DELETE /courses/1
```

RESPUESTA

```
{
  "message": "No se puede eliminar un recurso que todavia tiene matriculas",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

Un curso sin matriculas si:

PETICIÓN

```
DELETE /courses/2
```

RESPUESTA

```
{
  "title": "Introduccion a NestJS",
  "level": "beginner"
}
```

HTTP 200

Un curso con matriculas no se puede eliminar:

PETICIÓN

```
DELETE /courses/1
```

RESPUESTA

```
{
  "message": "No se puede eliminar un recurso que todavía tiene matrículas",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

Un estudiante con matriculas tampoco:

PETICIÓN

```
PATCH /students/1/status
Content-Type: application/json

{"isActive":true}
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software",
  "semester": 4,
  "isActive": true
}
```

HTTP 200

PETICIÓN

```
DELETE /students/1
```

RESPUESTA

```
{
  "message": "No se puede eliminar un recurso que todavía tiene matrículas",
  "error": "Conflict",
  "statusCode": 409
}
```

HTTP 409

SESION 10 - G - Reiniciar la API y cancelar la matricula

(La API fue detenida y vuelta a iniciar con npm run start)

PETICIÓN

```
GET /enrollments
```

RESPUESTA

```
[
  {
    "id": 1,
    "studentId": 1,
    "courseId": 1,
    "student": {
      "id": 1,
      "name": "Ana Torres",
      "career": "Software",
      "semester": 4,
      "isActive": true
    },
    "course": {
      "id": 1,
      "title": "Bases de datos relacionales",
      "level": "intermediate"
    }
  }
]
```

HTTP 200

PETICIÓN

```
DELETE /enrollments/1
```

RESPUESTA

```
{
  "id": 1,
  "studentId": 1,
  "courseId": 1,
  "student": {
    "id": 1,
    "name": "Ana Torres",
    "career": "Software",
    "semester": 4,
    "isActive": true
  },
  "course": {
    "id": 1,
    "title": "Bases de datos relacionales",
```

```
    "level": "intermediate"
  }
}
```

HTTP 200

Cancelar solo elimina el vinculo: el estudiante y el curso siguen ahi.

PETICIÓN

```
GET /students/1
```

RESPUESTA

```
{
  "id": 1,
  "name": "Ana Torres",
  "email": "ana.torres@example.edu",
  "age": 20,
  "career": "Software",
  "semester": 4,
  "isActive": true
}
```

HTTP 200

PETICIÓN

```
GET /courses/1
```

RESPUESTA

```
{
  "id": 1,
  "title": "Bases de datos relacionales",
  "level": "intermediate"
}
```

HTTP 200

PETICIÓN

```
GET /enrollments
```

RESPUESTA

```
[]
```

HTTP 200

Y ahora que no tiene matriculas, el curso si se puede eliminar:

PETICIÓN

```
DELETE /courses/1
```

RESPUESTA

```
{
  "title": "Bases de datos relacionales",
  "level": "intermediate"
}
```

HTTP 200

SESION 11 - FILTROS, PAGINACION Y ORDENAMIENTO EN / courses

Se sembraron 8 cursos con titulos y niveles distintos, como pide la sesion: una lista de uno o dos registros no permite comprobar una pagina real.

NOTA SOBRE EL PUERTO: el 3000 y el 3001 estaban ocupados por otro proyecto de la maquina, asi que esta demostracion se ejecuto con PORT=3333. El contrato de la API es identico.

Resumen

```
-------
A  Contrato completo de la sesion ......................... 200
B  Sin parametros usa los valores por defecto ............. 200
C  Tres paginas consecutivas sin repetir .................. 200
D  Ordenamiento descendente ............................... 200
E  Cinco parametros invalidos ............................. 400
```

SESION 11 - A - Contrato completo de la sesion

PETICIÓN

```
GET /courses?level=beginner&page=1&limit=3&sortBy=title&order=ASC
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "title": "Algoritmos y estructuras",
      "level": "beginner"
    },
    {
      "id": 5,
      "title": "Expresiones regulares",
      "level": "beginner"
    },
    {
      "id": 6,
      "title": "Fundamentos de TypeScript",
      "level": "beginner"
    }
  ],
  "meta": {
    "total": 4,
    "page": 1,
    "limit": 3,
    "totalPages": 2
  }
}
```


HTTP 200

SESION 11 - B - Sin parametros usa los valores por defecto

PETICIÓN

```
GET /courses
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "title": "Algoritmos y estructuras",
      "level": "beginner"
    },
    {
      "id": 2,
      "title": "Bases de datos relacionales",
      "level": "intermediate"
    },
    {
      "id": 3,
      "title": "Clean Architecture",
      "level": "advanced"
    },
    {
      "id": 4,
      "title": "Desarrollo con NestJS",
      "level": "intermediate"
    },
    {
      "id": 5,
      "title": "Expresiones regulares",
      "level": "beginner"
    },
    {
      "id": 6,
      "title": "Fundamentos de TypeScript",
      "level": "beginner"
    },
    {
      "id": 7,
      "title": "GraphQL en profundidad",
      "level": "advanced"
    },
    {
      "id": 8,
      "title": "HTTP y APIs REST",
      "level": "beginner"
    }
  ],
  "meta": {
    "total": 8,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```


HTTP 200

SESION 11 - C - Dos paginas consecutivas sin repetir

PETICIÓN

```
GET /courses?page=1&limit=3&sortBy=title&order=ASC
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "title": "Algoritmos y estructuras",
      "level": "beginner"
    },
    {
      "id": 2,
      "title": "Bases de datos relacionales",
      "level": "intermediate"
    },
    {
      "id": 3,
      "title": "Clean Architecture",
      "level": "advanced"
    }
  ],
  "meta": {
    "total": 8,
    "page": 1,
    "limit": 3,
    "totalPages": 3
  }
}
```


HTTP 200

PETICIÓN

```
GET /courses?page=2&limit=3&sortBy=title&order=ASC
```

RESPUESTA


```
{
  "items": [
    {
      "id": 4,
      "title": "Desarrollo con NestJS",
      "level": "intermediate"
    },
    {
      "id": 5,
      "title": "Expresiones regulares",
      "level": "beginner"
    },
    {
      "id": 6,
      "title": "Fundamentos de TypeScript",
      "level": "beginner"
    }
  ],
  "meta": {
    "total": 8,
```

```
    "page": 2,
    "limit": 3,
    "totalPages": 3
  }
}
```


HTTP 200

PETICIÓN

```
GET /courses?page=3&limit=3&sortBy=title&order=ASC
```

RESPUESTA


```
{
  "items": [
    {
      "id": 7,
      "title": "GraphQL en profundidad",
      "level": "advanced"
    },
    {
      "id": 8,
      "title": "HTTP y APIs REST",
      "level": "beginner"
    }
  ],
  "meta": {
    "total": 8,
    "page": 3,
    "limit": 3,
    "totalPages": 3
  }
}
```


HTTP 200

SESION 11 - D - Ordenamiento descendente

PETICIÓN

```
GET /courses?limit=3&sortBy=title&order=DESC
```

RESPUESTA


```
{
  "items": [
    {
      "id": 8,
      "title": "HTTP y APIs REST",
      "level": "beginner"
    },
    {
      "id": 7,
      "title": "GraphQL en profundidad",
      "level": "advanced"
    },
    {
      "id": 6,
      "title": "Fundamentos de TypeScript",
      "level": "beginner"
    }
  ],
  "meta": {
```

```
    "total": 8,
    "page": 1,
    "limit": 3,
    "totalPages": 3
  }
}
```

HTTP 200

SESION 11 - E - Parametros invalidos (400)

PETICIÓN

```
GET /courses?page=0
```

RESPUESTA

```
{
  "message": [
    "page must not be less than 1"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

HTTP 400

PETICIÓN

```
GET /courses?limit=200
```

RESPUESTA

```
{
  "message": [
    "limit must not be greater than 50"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

HTTP 400

PETICIÓN

```
GET /courses?sortBy=createdAt
```

RESPUESTA

```
{
  "message": [
    "sortBy must be one of the following values: id, title, level"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

HTTP 400

PETICIÓN

```
GET /courses?order=sideways
```

RESPUESTA

```
{
  "message": [
    "order must be one of the following values: ASC, DESC"
  ],
  "error": "Bad Request",
  "statusCode": 400
```

```
}
```

HTTP 400

PETICIÓN

```
GET /courses?pagina=2
```

RESPUESTA

```
{
  "message": [
    "property pagina should not exist"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

HTTP 400

Observaciones

```
-------------
```

En el caso A, meta.total vale 4 mientras items trae 3 elementos. Esa diferencia es justamente lo que permite al cliente saber que existe una segunda pagina sin pedir todos los datos.

En el caso C las tres paginas devuelven 8 cursos distintos, sin repeticiones ni ausencias, porque el ordenamiento por title es estable. Sin ordenamiento explicito PostgreSQL puede devolver las filas en cualquier orden y un mismo curso podria aparecer en dos paginas.

En el caso E, sortBy=createdAt se rechaza aunque sea un nombre plausible: solo se admiten id, title y level. El nombre de una columna no puede parametrizarse en la consulta, por eso se compara contra una lista cerrada en lugar de pasarlo directamente.

SESION 12 - BUSQUEDA Y CONSULTAS RELACIONADAS CON QUERYBUILDER

Datos de prueba: 4 cursos, 4 estudiantes y 6 matriculas. Los nombres se eligieron para probar coincidencias parciales: Ana, Anabel y el curso Analisis de APIs comparten el prefijo "ana". Bruno Diaz se desactivo despues de matricularse, para que activeOnly tenga algo que excluir.

Resumen

```
-------
A  Busqueda textual con ILIKE ............................ 200
B  Filtros combinables ..................................... 200
C  Busqueda sin resultados: 200 y no 404 .................. 200
D  Inyeccion SQL tratada como texto ....................... 200
E  Paginacion sin repetir matriculas ...................... 200
F  Parametros invalidos ................................... 400
```

SESION 12 - A - Busqueda textual con ILIKE

search=nest coincide con dos titulos distintos:

PETICIÓN

```
GET /enrollments?search=nest
```

RESPUESTA


```
{
  "items": [
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
```

```
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 6,
      "studentId": 3,
      "courseId": 4,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 4,
        "title": "Testing en NestJS",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
    "total": 3,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

No distingue mayusculas: NEST devuelve lo mismo.

PETICIÓN

```
GET /enrollments?search=NEST
```

RESPUESTA


```
{
  "items": [
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
```

```
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 6,
      "studentId": 3,
      "courseId": 4,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 4,
        "title": "Testing en NestJS",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
    "total": 3,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

search=ana coincide por NOMBRE, con Ana y tambien con Anabel:

PETICIÓN

```
GET /enrollments?search=ana
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "studentId": 1,
      "courseId": 1,
      "student": {
        "id": 1,
        "name": "Ana Torres",
```

```
      "career": "Software",
      "semester": 5,
      "isActive": true
    },
    "course": {
      "id": 1,
      "title": "Analisis de APIs",
      "level": "intermediate"
    }
  },
  {
    "id": 2,
    "studentId": 1,
    "courseId": 2,
    "student": {
      "id": 1,
      "name": "Ana Torres",
      "career": "Software",
      "semester": 5,
      "isActive": true
    },
    "course": {
      "id": 2,
      "title": "Desarrollo con NestJS",
      "level": "beginner"
    }
  },
  {
    "id": 3,
    "studentId": 2,
    "courseId": 2,
    "student": {
      "id": 2,
      "name": "Anabel Ruiz",
      "career": "Software",
      "semester": 5,
      "isActive": true
    },
    "course": {
      "id": 2,
      "title": "Desarrollo con NestJS",
      "level": "beginner"
    }
  },
  {
    "id": 4,
    "studentId": 2,
    "courseId": 3,
    "student": {
      "id": 2,
      "name": "Anabel Ruiz",
      "career": "Software",
      "semester": 5,
      "isActive": true
    },
    "course": {
      "id": 3,
```

```
        "title": "Bases de datos",
        "level": "advanced"
      }
    },
    {
      "id": 5,
      "studentId": 3,
      "courseId": 1,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
    "total": 5,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

SESION 12 - B - Filtros combinables

Solo estudiantes activos (Bruno quedo inactivo):

PETICIÓN

```
GET /enrollments?activeOnly=true
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "studentId": 1,
      "courseId": 1,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
```

```
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 4,
      "studentId": 2,
      "courseId": 3,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 3,
        "title": "Bases de datos",
        "level": "advanced"
      }
    }
  ],
  "meta": {
    "total": 4,
    "page": 1,
    "limit": 10
  }
```

```
}
```

HTTP 200

activeOnly=false NO se interpreta como verdadero:

PETICIÓN

```
GET /enrollments?activeOnly=false
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "studentId": 1,
      "courseId": 1,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
```

```
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 4,
      "studentId": 2,
      "courseId": 3,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 3,
        "title": "Bases de datos",
        "level": "advanced"
      }
    },
    {
      "id": 5,
      "studentId": 3,
      "courseId": 1,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
    {
      "id": 6,
      "studentId": 3,
      "courseId": 4,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 4,
        "title": "Testing en NestJS",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
```

```
    "total": 6,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

Busqueda y filtro de actividad combinados, sin sobrescribirse:

PETICIÓN

```
GET /enrollments?search=nest&activeOnly=true&page=1&limit=5
```

RESPUESTA


```
{
  "items": [
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    }
  ],
  "meta": {
    "total": 2,
    "page": 1,
    "limit": 5
  }
}
```


HTTP 200

Filtro por curso:

PETICIÓN

```
GET /enrollments?courseId=1
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "studentId": 1,
      "courseId": 1,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
    {
      "id": 5,
      "studentId": 3,
      "courseId": 1,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
    "total": 2,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

Filtro por estudiante mas busqueda:

PETICIÓN

```
GET /enrollments?studentId=1&search=nest
```

RESPUESTA


```
{
  "items": [
```

```
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    }
  ],
  "meta": {
    "total": 1,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

SESION 12 - C - Busqueda sin resultados: 200, no 404

PETICIÓN

```
GET /enrollments?search=inexistente
```

RESPUESTA


```
{
  "items": [],
  "meta": {
    "total": 0,
    "page": 1,
    "limit": 10
  }
}
```


HTTP 200

SESION 12 - D - Inyeccion SQL tratada como texto

Si el texto se interpolara, 1=1 devolveria las 6 matriculas.

Al ir como parametro, se busca literalmente y no coincide nada.

PETICIÓN

```
GET /enrollments?search=%27%20OR%201%3D1%20--
```

RESPUESTA


```
{
  "items": [],
  "meta": {
    "total": 0,
    "page": 1,
    "limit": 10
```

```
  }
}
```

HTTP 200

PETICIÓN

```
GET /enrollments?search=%25
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "studentId": 1,
      "courseId": 1,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
```

```
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 4,
      "studentId": 2,
      "courseId": 3,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 3,
        "title": "Bases de datos",
        "level": "advanced"
      }
    },
    {
      "id": 5,
      "studentId": 3,
      "courseId": 1,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
    {
      "id": 6,
      "studentId": 3,
      "courseId": 4,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 4,
        "title": "Testing en NestJS",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
```

```
    "total": 6,
    "page": 1,
    "limit": 10
  }
}
```

HTTP 200

SESION 12 - E - Paginacion: cada matricula una sola vez

PETICIÓN

```
GET /enrollments?page=1&limit=3
```

RESPUESTA


```
{
  "items": [
    {
      "id": 1,
      "studentId": 1,
      "courseId": 1,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
    },
    {
      "id": 2,
      "studentId": 1,
      "courseId": 2,
      "student": {
        "id": 1,
        "name": "Ana Torres",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    },
    {
      "id": 3,
      "studentId": 2,
      "courseId": 2,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
```

```
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 2,
        "title": "Desarrollo con NestJS",
        "level": "beginner"
      }
    }
  ],
  "meta": {
    "total": 6,
    "page": 1,
    "limit": 3
  }
}
```


HTTP 200

PETICIÓN

```
GET /enrollments?page=2&limit=3
```

RESPUESTA


```
{
  "items": [
    {
      "id": 4,
      "studentId": 2,
      "courseId": 3,
      "student": {
        "id": 2,
        "name": "Anabel Ruiz",
        "career": "Software",
        "semester": 5,
        "isActive": true
      },
      "course": {
        "id": 3,
        "title": "Bases de datos",
        "level": "advanced"
      }
    },
    {
      "id": 5,
      "studentId": 3,
      "courseId": 1,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 1,
        "title": "Analisis de APIs",
        "level": "intermediate"
      }
```

```
    },
    {
      "id": 6,
      "studentId": 3,
      "courseId": 4,
      "student": {
        "id": 3,
        "name": "Bruno Diaz",
        "career": "Software",
        "semester": 5,
        "isActive": false
      },
      "course": {
        "id": 4,
        "title": "Testing en NestJS",
        "level": "intermediate"
      }
    }
  ],
  "meta": {
    "total": 6,
    "page": 2,
    "limit": 3
  }
}
```


HTTP 200

SESION 12 - F - Parametros invalidos

PETICIÓN

```
GET /enrollments?activeOnly=quizas
```

RESPUESTA


```
{
  "message": [
    "activeOnly must be a boolean string"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```


HTTP 400

PETICIÓN

```
GET /enrollments?limit=999
```

RESPUESTA


```
{
  "message": [
    "limit must not be greater than 50"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```


HTTP 400

Observaciones

```
-------------
```

En A, search=ana devuelve 5 matriculas porque el ILIKE se aplica al nombre del estudiante y al titulo del curso dentro de un mismo parentesis: coincide con Ana Torres, con Anabel Ruiz y con el curso Analisis de APIs. Los parentesis importan, porque sin ellos el OR se mezclaria con los demas filtros y los sobrescribiria.

En B, activeOnly=false devuelve las 6 matriculas y no 0. El servicio compara con la cadena 'true' a proposito: Boolean('false') en JavaScript es true, de modo que convertir el texto directamente invertiria el significado del filtro.

En C, una busqueda sin coincidencias responde 200 con items vacio. El recurso "coleccion de matriculas" existe aunque ninguna fila coincida; un 404 afirmaria que la coleccion no existe, que es falso.

En D, la entrada ' OR 1=1 -- devuelve total 0. Si el texto se concatenara dentro del SQL, esa condicion seria siempre verdadera y devolveria las 6 matriculas. Al viajar como parametro :search, se busca literalmente ese texto y no coincide con ningun nombre ni titulo.

Un matiz honesto sobre la misma prueba: search=% si devuelve las 6 matriculas, porque el signo de porcentaje es un comodin dentro del patron LIKE. Eso no es inyeccion: el cliente no puede alterar la estructura de la consulta ni leer otras tablas, solo ampliar su propia busqueda. Si el producto necesitara impedirlo, habria que escapar los comodines en el valor antes de armar el patron.